REPORT

Algorithm	Execution Time (ms)	Packet Length (bits)	Key Length (bits)
AES-128-CBC-ENC	0.16117095947265625	1152	128
AES-128-CBC-DEC	0.0820159912109375		
AES-128-CTR-ENC	0.07200241088867188	1040	128
AES-128-CTR-DEC	0.03743171691894531		
RSA-2048-ENC	1.2106895446777344	2176	13632
RSA-2048-DEC	36.664724349975586		
AES-128-CMAC-GEN	0.06008148193359375	1296	128
AES-128-CMAC-VRF	0.03361701965332031		
SHA3-256-HMAC-GEN	0.03361701965332031	1296	128
SHA3-256-HMAC-VRF	0.0209808349609375		
RSA-2048-SHA3-256-SIG-GEN	37.29677200317383	3792	3608
RSA-2048-SHA3-256-SIG-VRF	0.9782314300537109		
ECDSA-256-SHA3-256-SIG-GEN	1.0128021240234375	1808	1424
ECDSA-256-SHA3-256-SIG-VRF	0.9701251983642578		
AES-128-GCM-GEN	0.10180473327636719	142	128
AES-128-GCM-VRF	0.04553794860839844		

AES-128-CBC Encryption/Decryption:
Pros: The symmetric encryption algorithm is widely supported and used. offers confidentiality and robust encryption.
Cons: If not executed with caution, susceptible to padding oracle attacks. adds overhead by requiring an initialization vector (IV) for every encryption.
AES-128-CTR Encryption/Decryption:
Pros: You can access encrypted data at random without having to decrypt the complete ciphertext. unable to be padded by oracle attacks.
Cons: Increases complexity by requiring a different nonce for every encryption.
RSA-2048 Encryption/Decryption:
Pros: Secure communication across unsecure channels is made possible by asymmetric encryption. maintains key exchange, confidentiality, and authenticity.
Cons: It operates more slowly than symmetric encryption. restricted to brief texts because of key size limitations.

AES-128-CMAC Generation:
Pros: Uses message authentication codes (MACs) to preserve integrity. resistant to forgeries and message manipulation.
Cons: It's not as popular as HMAC. The management of keys is essential to security.
SHA3-256-HMAC Generation:
Pros: Robust and effective message authentication code based on hashing. able to withstand collisional strikes.
Cons: Careful key handling is necessary. If not executed appropriately, susceptible to attacks involving length extension.
RSA-2048-SHA3-256 Signature Generation:
Pros: Non-repudiation is possible with digital signatures. Enables message integrity and sender identification verification.
Cons: It moves more slowly than symmetric algorithms. Complexity of key management. restricted to brief texts because of key size limitations.
ECDSA-256-SHA3-256 Signature Generation:
Pros: Offers integrity verification and non-repudiation. more effective at creating signatures than RSA.
Cons: Complexity of key management. restricted to brief texts because of key size limitations.

AES-128-CBC Encryption: def aes_128_cbc_encrypt(key: bytes, iv: bytes, plaintext: str) -> str:
AES-128-CBC Decryption: def aes_128_cbc_decrypt(key: bytes, iv: bytes, ciphertext: str) -> str:
AES-128-CTR Encryption: def aes_128_ctr_encrypt(key: bytes, nonce: bytes, plaintext: str) -> str:
AES-128-CTR Decryption: def aes_128_ctr_decrypt(key: bytes, nonce: bytes, ciphertext: str) -> str:
RSA-2048 Encryption: def rsa_2048_encrypt(public_key: str, plaintext: str) -> str:
RSA-2048 Decryption: def rsa_2048_decrypt(private_key: str, ciphertext: str) -> bytes:
AES-128-CMAC : def generate_aes_cmac(key: bytes, plaintext: str) -> bytes:
SHA3-256-HMAC : def generate_sha3_hmac(key: bytes, plaintext: str) -> bytes:
RSA-2048-SHA3-256 Signature : def generate_rsa_signature(private_key: str, plaintext: str) -> str:
ECDSA-256-SHA3-256 Signature : def generate_ecdsa_signature(private_key: str, plaintext: str) -> str:

Prototype:

```python
def aes_128_cbc_encrypt(key, iv, plaintext):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()

    # Use PKCS7 directly from cryptography.hazmat.primitives.padding
    padder = PKCS7(algorithms.AES.block_size).padder()

    padded_data = padder.update(plaintext.encode('utf-8')) + padder.finalize()
    ciphertext = encryptor.update(padded_data) + encryptor.finalize()
    return b64encode(ciphertext).decode('utf-8')
```

```
def aes_128_cbc_decrypt(key, iv, ciphertext):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()

    # Use PKCS7 directly from cryptography.hazmat.primitives.padding
    unpadder = PKCS7(algorithms.AES.block_size).unpadder()

    decrypted_data = decryptor.update(b64decode(ciphertext)) + decryptor.finalize()
    plaintext = unpadder.update(decrypted_data) + unpadder.finalize()
    return plaintext.decode('utf-8')
```

```
def aes_128_ctr_encrypt(key, nonce, plaintext):
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce), backend=default_backend())
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext.encode('utf-8')) + encryptor.finalize()
    return b64encode(ciphertext).decode('utf-8')
```

```python
def aes_128_ctr_decrypt(key, nonce, ciphertext):
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce), backend=default_backend())
    decryptor = cipher.decryptor()
    decrypted_data = decryptor.update(b64decode(ciphertext)) + decryptor.finalize()
    return decrypted_data.decode('utf-8')
```

```python
def rsa_2048_encrypt(public_key, plaintext):
    key = serialization.load_pem_public_key(public_key.encode('utf-8'), backend=default_backend())
    if isinstance(plaintext, str):
        plaintext = plaintext.encode('utf-8')

    ciphertext = key.encrypt(
        plaintext,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return b64encode(ciphertext).decode('utf-8')
```

```python
def rsa_2048_decrypt(private_key, ciphertext):
    key = serialization.load_pem_private_key(
        private_key.encode('utf-8'),
        password=None,
        backend=default_backend()
    )
    decrypted_data = key.decrypt(
        b64decode(ciphertext),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return decrypted_data
```